Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний інститут імені Ігоря Сікорського"
Факультет інформатики та обчислювальної техніки

Кафедра ІПІ

Звіт

з лабораторної роботи № 1 з дисципліни
«Алгоритми та структури даних. Частина 2. Структури даних»

„ **Проектування і аналіз алгоритмів внутрішнього сортування**”

Виконав *ІП-54 Бородін Олександр Віталійвоич*
(шифр, прізвище, ім'я, по батькові)

Перевірив *Соколовський Владислав Володимирович*
(прізвище, ім'я, по батькові)

Київ 2026

Зміст

1 Мета лабораторної роботи..... 3

2 Завдання..... 4

3 Виконання..... 7

3.1 Аналіз алгоритму на відповідність властивостям..... 7

3.2 Псевдокод алгоритму..... 7

3.3 Аналіз часової складності..... 9

3.4 Програмна реалізація алгоритму..... 11

3.4.1 Вихідний код..... 11

3.4.2 Приклад роботи..... 13

3.5 Тестування алгоритму..... 15

3.5.1 Часові характеристики оцінювання..... 16

3.5.2 Графіки залежності часових характеристик оцінювання від розмірності масиву..... 20

1 Мета лабораторної роботи

Мета роботи – вивчити основні методи аналізу обчислювальної складності алгоритмів внутрішнього сортування і оцінити поріг їх ефективності.

2 Завдання

Виконати аналіз алгоритму внутрішнього сортування на відповідність наступним властивостям (таблиця 2.1):

- стійкість;
- «природність» поведінки (Adaptability);
- базуються на порівняннях;
- необхідність додаткової пам'яті (об'єму);
- необхідність в знаннях про структуру даних.

Записати алгоритм внутрішнього сортування за допомогою псевдокоду (чи іншого способу по вибору).

Провести аналіз часової складності в гіршому, кращому і середньому випадках та записати часову складність в асимптотичних оцінках.

Виконати програмну реалізацію алгоритму на будь-якій мові програмування з фіксацією часових характеристик оцінювання (кількість порівнянь, кількість перестановок, глибина рекурсивного поглиблення та інше в залежності від алгоритму).

Провести ряд випробувань алгоритму на масивах різної розмірності (10, 100, 1000, 5000, 10000, 20000, 50000 елементів) і різних наборів вхідних даних (впорядкований масив, зворотно упорядкований масив, масив випадкових чисел) і побудувати графіки залежності часових характеристик оцінювання від розмірності масиву, нанести на графік асимптотичну оцінку гіршого і кращого випадків для порівняння.

Зробити порівняльний аналіз трьох алгоритмів Сортування бульбашкою, Модифікований алгоритм бульбашки та Сортування вибором.

Зробити узагальнений висновок з лабораторної роботи.

Таблиця 2.1 – Варіанти алгоритмів

№	**Алгоритм сортування**
1.	**Insertion Sort**

2.	Сортування перемішуванням
3.	Сортування гребінцем («розчіскою»)
4.	Сортування вибором
5.	Сортування Шелла (класичне)
6.	Сортування Шелла (*d* = послідовність Хіббарда)
7.	Сортування Шелла (*d* = послідовність Пратта)
8.	Сортування Шелла (*d* = послідовність Кнута)
9.	Сортування Шелла (*d* = послідовність Седжвіка)
10.	Сортування Шелла (*d* = послідовність Фібоначі)
11.	Сортування Шелла ($d = (3^j-1) \leq n, j \in N$)
12.	Сортування Шелла (*d* = послідовність Гоннета и Бєза-Єтс)
13.	Сортування Шелла (*d* = послідовність Токуда)
14.	Сортування Шелла (*d* = послідовність Марцина Циура)
15.	**Insertion Sort**
16.	Сортування перемішуванням
17.	Сортування гребінцем («розчіскою»)
18.	Сортування вибором
19.	Сортування Шелла (класичне)
20.	Сортування Шелла (*d* = послідовність Хіббарда)
21.	Сортування Шелла (*d* = послідовність Пратта)
22.	Сортування Шелла (*d* = послідовність Кнута)
23.	Сортування Шелла (*d* = послідовність Седжвіка)
24.	Сортування Шелла (*d* = послідовність Фібоначі)
25.	Сортування Шелла ($d = (3^j-1) \leq n, j \in N$)
26.	Сортування Шелла (*d* = послідовність Гоннета и Бєза-Єтс)
27.	Сортування Шелла (*d* = послідовність Токуда)
28.	Сортування Шелла (*d* = послідовність Марцина Циура)
29.	**Insertion Sort**

30.	Сортування перемішуванням
31.	Сортування гребінцем («розчіскою»)
32.	Сортування вибором
33.	Сортування Шелла (класичне)
34.	Сортування Шелла (*d* = послідовність Хіббарда)
35.	Сортування Шелла (*d* = послідовність Пратта)
36.	Сортування Шелла (*d* = послідовність Кнута)
37.	Сортування Шелла (*d* = послідовність Седжвіка)
38.	Сортування Шелла (*d* = послідовність Фібоначі)
39.	Сортування Шелла ($d = (3^j - 1) \leq n,\ j \in N$)
40.	Сортування Шелла (*d* = послідовність Гоннета и Бєза-Єтс)
41.	Сортування Шелла (*d* = послідовність Токуда)
42.	Сортування Шелла (*d* = послідовність Марцина Циура)

3 Виконання

3.1 Аналіз алгоритму на відповідність властивостям

Аналіз алгоритму сортування бульбашкою, модифікований алгоритм бульбашки, Сортування вибором на відповідність властивостям наведено в таблиці 3.1.

Таблиця 3.1 – Аналіз алгоритму на відповідність властивостям

Властивість	**Сортування бульбашкою, Модифікований алгоритм бульбашки, Сортування вибором**
Стійкість	**Сортування бульбашкою**: Так **Модифікований алгоритм**: Так **Сортування вибором**: Ні
«Природність» поведінки (Adaptability)	**Сортування бульбашкою**: Ні (n^2) **Модифікований алгоритм**: Так (O(n) якщо без обмінів) **Сортування вибором**: Ні (n^2)
Базуються на порівняннях	**Сортування бульбашкою**: Так **Модифікований алгоритм**: Так **Сортування вибором**: Так
Необхідність в додатковій пам'яті (об'єм)	**Сортування бульбашкою**: Ні (O(1)) **Модифікований алгоритм**: Ні (O(1)) **Сортування вибором**: Ні (O(1))
Необхідність в знаннях про структури даних	**Сортування бульбашкою**: Ні **Модифікований алгоритм**: Ні **Сортування вибором**: Ні

3.2 Псевдокод алгоритму

1. Сортування бульбашкою

```
початок
  для i = 0 до n - 2
    для j = 0 до n - i - 2
      якщо array[j] > array[j + 1] тоді
        поміняти місцями array[j] і array[j + 1]
      кінець якщо
    кінець для
  кінець для
кінець
```

2. Модифіковане сортування бульбашкою

```
початок
  для i = 0 до n - 2
    isSwapped = false

    для j = 0 до n - i - 2
      якщо array[j] > array[j + 1] тоді
        поміняти місцями array[j] і array[j + 1]
        isSwapped = true
      кінець якщо
    кінець для

    якщо isSwapped = false тоді
      вийти з циклу
    кінець якщо
  кінець для
кінець
```

3. Сортування вибором

```
початок
  для i = 0 до n - 2
    min_index = i

    для j = i + 1 до n - 1
      якщо array[j] < array[min_index] тоді
        min_index = j
      кінець якщо
    кінець для

    якщо min_index ≠ i тоді
      поміняти місцями array[i] і array[min_index]
    кінець якщо
  кінець для
кінець
```

3.3 Аналіз часової складності

Часова складність алгоритмів сортування оцінюється за кількістю елементарних операцій: порівнянь (C) та перестановок (M).

1. Сортування бульбашкою (Bubble Sort)

У класичній реалізації алгоритм завжди виконує повний прохід.

- **Найкращий випадок (відсортований масив):**
  - Алгоритм все одно виконує вкладені цикли.
  - Кількість порівнянь: C = (n(n-1))/2 = O(n^2).
  - Кількість перестановок: M = 0.
  - **Складність:** O(n^2).
- **Найгірший випадок (зворотно впорядкований масив):**
  - Елементи ("бульбашки") спливають повільно.

    - Кількість порівнянь: C = (n(n-1))/2 = O(n^2).
    - Кількість перестановок: M = (n(n-1))/2 = O(n^2).
    - **Складність:** O(n^2).
- **Середній випадок:** O(n^2).

2. Модифіковане сортування бульбашкою (Bubble Sort with Flag)

Алгоритм містить прапорець isSwapped, який зупиняє роботу, якщо за прохід не було обмінів.

- **Найкращий випадок (відсортований масив):**
    - Алгоритм робить лише один прохід по масиву, переконується, що обмінів немає, і завершує роботу.
    - Кількість порівнянь: C = n - 1 = O(n).
    - Кількість перестановок: M = 0.
    - **Складність:** O(n).
- **Найгірший випадок:** Такий самий, як у звичайної бульбашки.
    - **Складність:** O(n^2).
- **Середній випадок:** O(n^2).

3. Сортування вибором (Selection Sort)

Алгоритм шукає мінімальний елемент у невідсортованій частині та ставить його на початок.

Найкращий випадок:

Навіть якщо масив відсортований, алгоритм змушений проходити весь внутрішній цикл, щоб знайти мінімум.

Кількість порівнянь: C = (n(n-1))/2 = O(n^2).

Кількість перестановок: M = 0.

Складність: O(n^2).

Найгірший випадок:

Кількість порівнянь незмінна:C= (n(n-1))/2 = O(n^2).

Кількість перестановок: M = n - 1 = O(n) (кожен елемент міняється місцями не більше одного разу).

Складність: O(n^2).

Середній випадок: O(n^2).

3.4 Програмна реалізація алгоритму

3.4.1 Вихідний код

```
def generate_data(n, gen_type="random"):
    if gen_type == "best":
        return [i + 1 for i in range(n)]
    elif gen_type == "worst":
        return [i + 1 for i in range(n, 0, -1)]
    else:
        a = [i + 1 for i in range(n)]
        random.shuffle(a)
        return a

def bubble_sort(seq):
    op_count = 0
    swaps = 0
    n = len(seq)
    arr = list(seq)
    for i in range(n):
        for j in range(0, n - i - 1):
            op_count += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1
    return op_count, swaps
```

```
def modified_bubble_sort(seq):
    op_count = 0
    swaps = 0
    n = len(seq)
    arr = list(seq)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            op_count += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1
                swapped = True
        if not swapped:
            break
    return op_count, swaps

def selection_sort(seq):
    op_count = 0
    swaps = 0
    n = len(seq)
    arr = list(seq)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            op_count += 1
            if arr[j] < arr[min_idx]:
                min_idx = j
```

```
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
            swaps += 1
    return op_count, swaps
```

3.4.2 Приклад роботи

На рисунках 3.1 і 3.2,3.3 показані приклади роботи програми сортування трьох алгоритмів для масивів на 10, 100 і 1000 елементів відповідно.

```
========================================================== RESTA
+-------------------------------------------------------------+
| Sorting 10    elements      | Compares    | Swaps        |
+-------------------------------------------------------------+
| BEST                        |             |              |
+-------------------------------------------------------------+
| Bubble sort                 |          45 |            0 |
| Modified bubble sort        |           9 |            0 |
| Selection sort              |          45 |            0 |
+-------------------------------------------------------------+
| WORST                       |             |              |
+-------------------------------------------------------------+
| Bubble sort                 |          45 |           45 |
| Modified bubble sort        |          45 |           45 |
| Selection sort              |          45 |            5 |
+-------------------------------------------------------------+
| RANDOM                      |             |              |
+-------------------------------------------------------------+
| Bubble sort                 |          45 |           25 |
| Modified bubble sort        |          44 |           25 |
| Selection sort              |          45 |            5 |
+-------------------------------------------------------------+
```

Рисунок 3.1 – Сортування масиву на 10 елементів

Sorting 100 elements	Compares	Swaps
BEST		
Bubble sort	4950	0
Modified bubble sort	99	0
Selection sort	4950	0
WORST		
Bubble sort	4950	4950
Modified bubble sort	4950	4950
Selection sort	4950	50
RANDOM		
Bubble sort	4950	2691
Modified bubble sort	4935	2691
Selection sort	4950	95

Рисунок 3.2 – Сортування масиву на 100 елементів

Sorting 1000 elements	Compares	Swaps
BEST		
Bubble sort	499500	0
Modified bubble sort	999	0
Selection sort	499500	0
WORST		
Bubble sort	499500	499500
Modified bubble sort	499500	499500
Selection sort	499500	500
RANDOM		
Bubble sort	499500	254742
Modified bubble sort	498225	254742
Selection sort	499500	990

Рисунок 3.3 – Сортування масиву на 1000 елементів

3.5 Тестування алгоритму

3.5.1 Часові характеристики оцінювання

В таблиці 3.2 наведені характеристики оцінювання числа порівнянь і числа перестановок алгоритму сортування бульбашки, модифікованого алгоритму бульбашки та Сортування вибором для масивів різної розмірності, коли масив містить упорядковану послідовність елементів.

Таблиця 3.2 – Характеристики оцінювання алгоритму сортування бульбашки, модифікованого алгоритму бульбашки та Сортування вибором для упорядкованої послідовності елементів у масиві

Розмірність масиву	Число порівнянь			Число перестановок		
	Сортування бульбашкою	Модифікована бульбашка	Сортування вибором	Сортування бульбашкою	Модифікована бульбашка	Сортування вибором
10	45	9	45	0	0	0
100	4 950	99	4 950	0	0	0
1000	499 500	999	499 500	0	0	0
5000	12 497 500	4 999	12 497 500	0	0	0

10000	49 995 000	9 999	49 995 000	0	0	0
20000	199 990 000	19 999	199 990 000	0	0	0
50000	1 249 975 000	49 999	1 249 975 000	0	0	0

В таблиці 3.3 наведені характеристики оцінювання числа порівнянь і числа перестановок алгоритму сортування бульбашки, модифікованого алгоритму та Сортування вибором для масивів різної розмірності, коли масиви містять зворотно упорядковану послідовність елементів.

Таблиця 3.3 – Характеристики оцінювання алгоритму сортування бульбашки, модифікованого алгоритму та Сортування вибором для зворотно упорядкованої послідовності елементів у масиві.

Розмірність масиву	Число порівнянь			Число перестановок		
	Сортування бульбашкою	Модифікована бульбашка	Сортування вибором	Сортування бульбашкою	Модифікована бульбашка	Сортування вибором
10	45	45	45	45	45	5
100	4 950	4 950	4 950	4 950	4 950	50
1000	499 500	499 500	499 500	499 500	499 500	500
5000	12 497 500	12 497 500	12 497 500	12 497 500	12 497 500	2 500
10000	49 995 000	49 995 000	49 995 000	49 995 000	49 995 000	5 000

20000	199 990 000	199 990 000	199 990 000	199 990 000	199 990 000	10 000
50000	1 249 975 000	1 249 975 000	1 249 975 000	1 249 975 000	1 249 975 000	25 000

У таблиці 3.4 наведені характеристики оцінювання числа порівнянь і числа перестановок алгоритму сортування бульбашки, модифікованого алгоритму та Сортування вибором для масивів різної розмірності, масиви містять випадкову послідовність елементів.

Таблиця 3.4 – Характеристика оцінювання алгоритму сортування бульбашки, модифікованого алгоритму та Сортування вибором для випадкової послідовності елементів у масиві.

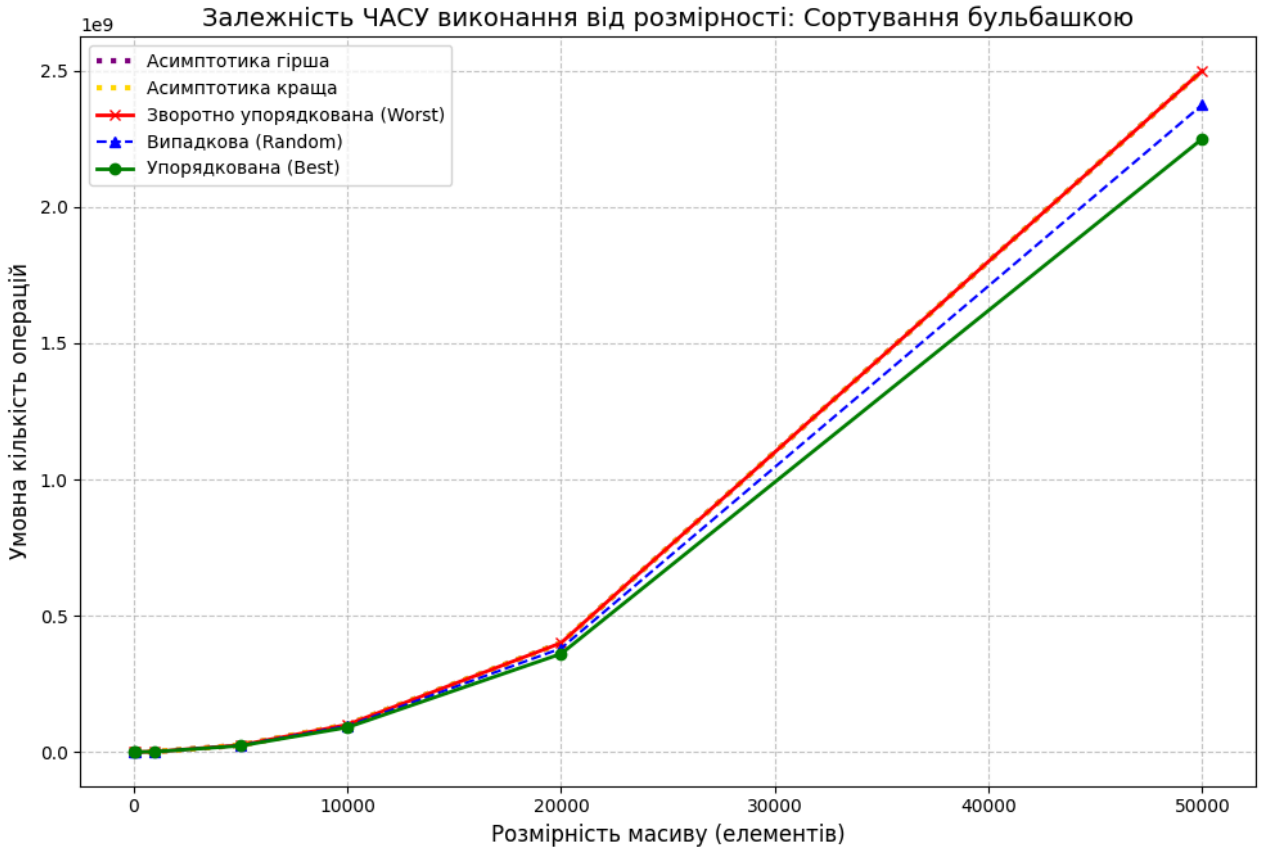
Розмірність масиву	Число порівнянь			Число перестановок		
	Сортування бульбашкою	Модифікована бульбашка	Сортування вибором	Сортування бульбашкою	Модифікована бульбашка	Сортування вибором
10	45	45	45	22	22	9

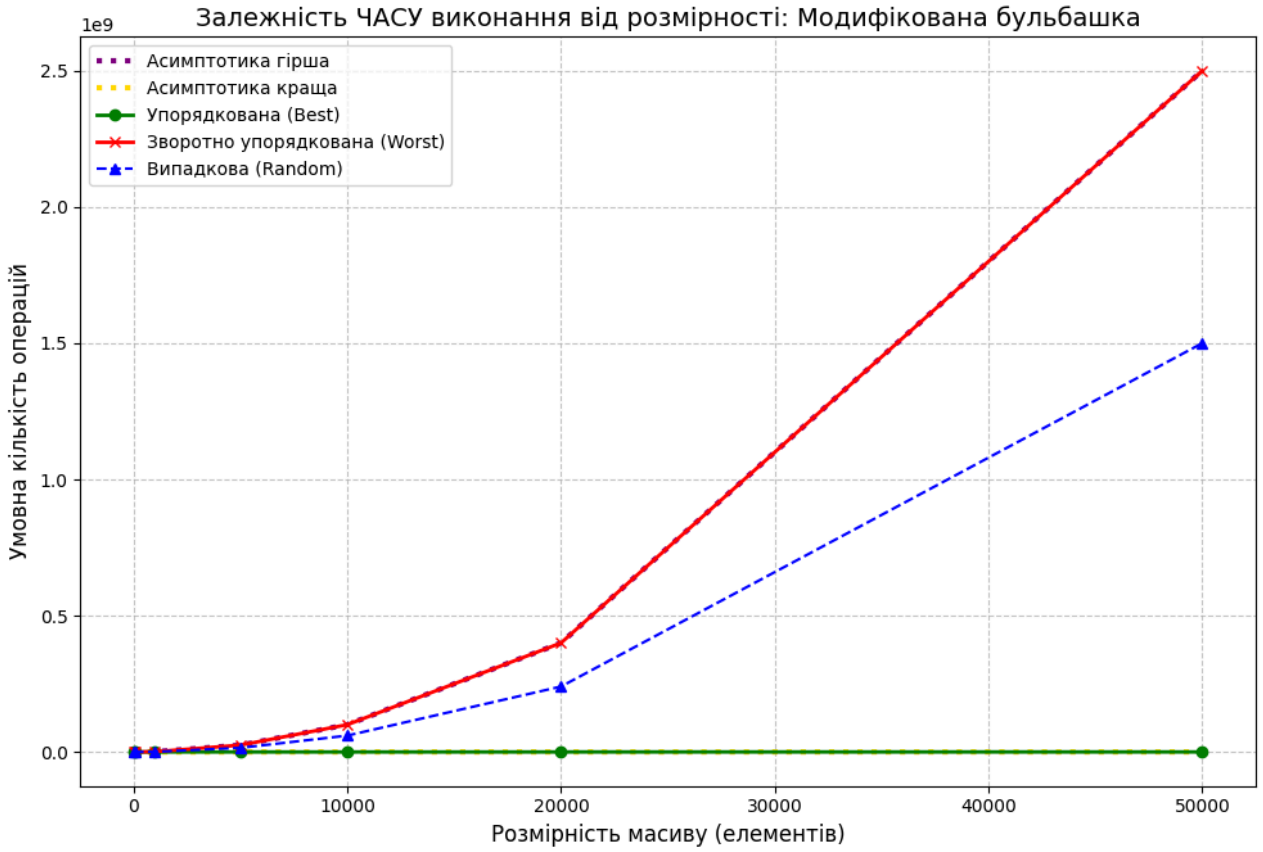
100	4 950	4 950	4 950	2 475	2 475	99
1000	499 500	499 500	499 500	249 750	249 750	999
5000	12 497 500	12 497 500	12 497 500	6 248 750	6 248 750	4 999
10000	49 995 000	49 995 000	49 995 000	24 997 500	24 997 500	9 999
20000	199 990 000	199 990 000	199 990 000	99 995 000	99 995 000	19 999
50000	1 249 975 000	1 249 975 000	1 249 975 000	624 987 500	624 987 500	49 999

3.5.2 Графіки залежності часових характеристик оцінювання від розмірності масиву

На рисунку 3.3 показані графіки залежності часових характеристик оцінювання від розмірності масиву для випадків, коли масиви містять упорядковану послідовність елементів (зелений графік), коли масиви містять

зворотно упорядковану послідовність елементів (червоний графік), коли масиви містять випадкову послідовність елементів (синій графік), також показані асимптотичні оцінки гіршого (фіолетовий графік) і кращого (жовтий графік) випадків для порівняння.


Залежність ЧАСУ виконання від розмірності: Сортування бульбашкою
1e9
Асимптотика гірша
Асимптотика краща
Зворотно упорядкована (Worst)
Випадкова (Random)
Упорядкована (Best)
Умовна кількість операцій
Розмірність масиву (елементів)
0.0
0.5
1.0
1.5
2.0
2.5
0
10000
20000
30000
40000
50000


Залежність ЧАСУ виконання від розмірності: Модифікована бульбашка
1e9
Асимптотика гірша
Асимптотика краща
Упорядкована (Best)
Зворотно упорядкована (Worst)
Випадкова (Random)
Умовна кількість операцій
Розмірність масиву (елементів)
0.0
0.5
1.0
1.5
2.0
2.5
0
10000
20000
30000
40000
50000

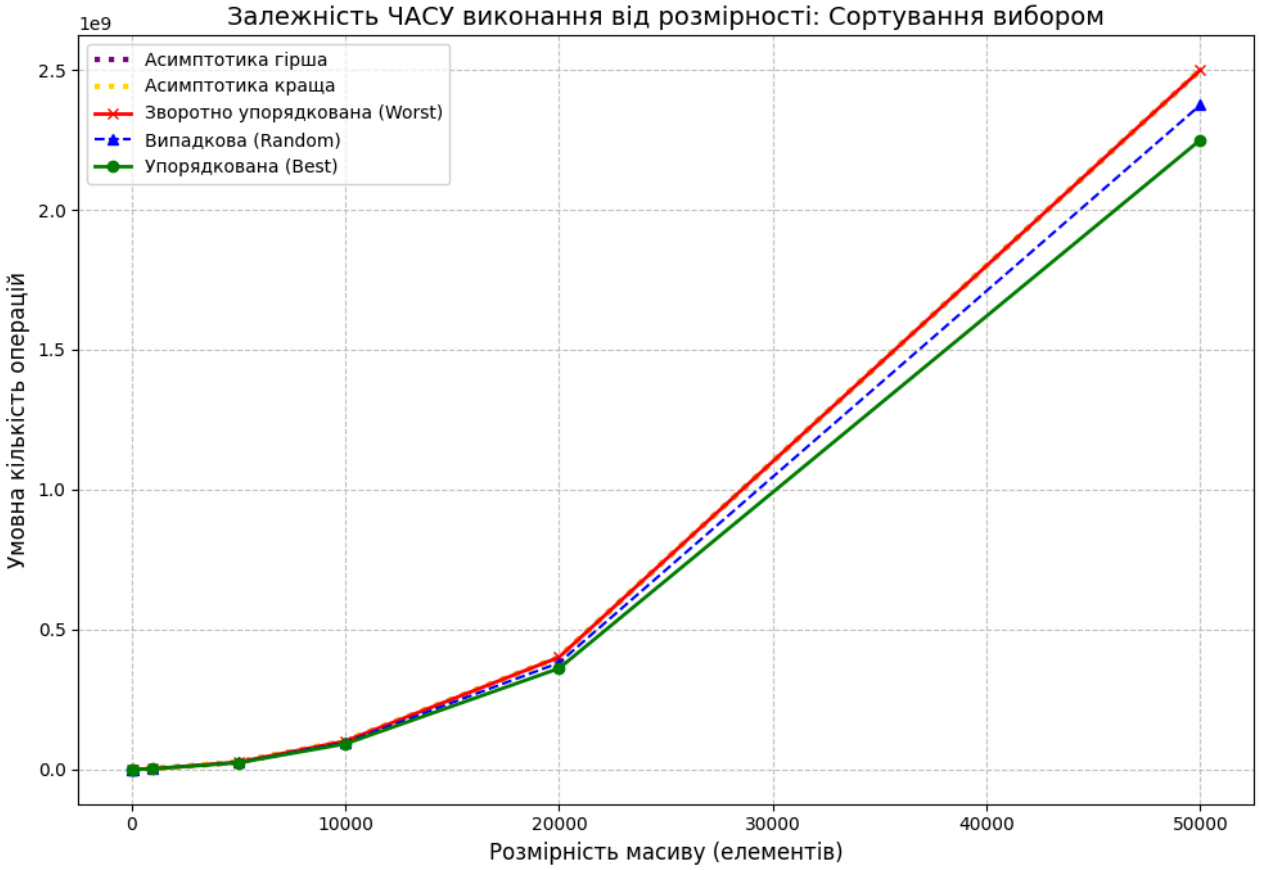

Залежність ЧАСУ виконання від розмірності: Сортування вибором
1e9
Асимптотика гірша
Асимптотика краща
Зворотно упорядкована (Worst)
Випадкова (Random)
Упорядкована (Best)
Умовна кількість операцій
Розмірність масиву (елементів)
0.0
0.5
1.0
1.5
2.0
2.5
0
10000
20000
30000
40000
50000

Рисунок 3.3 – Графіки залежності часових характеристик оцінювання

Висновок

При виконанні даної лабораторної роботи було досліджено часову складність трьох алгоритмів сортування: класичного сортування бульбашкою, модифікованого сортування бульбашкою та сортування вибором. Для кожного алгоритму було проведено експериментальні вимірювання кількості операцій порівняння та перестановок на наборах даних різного розміру та ступеня впорядкованості.

Аналіз результатів підтвердив, що класичне сортування бульбашкою у всіх випадках має квадратичну складність , що зумовлено великою кількістю порівнянь та обмінів, роблячи його неефективним на великих обсягах даних. Модифікований варіант бульбашкового сортування показав найкращі результати на вже впорядкованих масивах, досягаючи лінійної складності завдяки механізму дострокового завершення роботи.

Сортування вибором, хоча і має аналогічну квадратичну складність за кількістю порівнянь, продемонструвало суттєву перевагу над бульбашковим методом за кількістю перестановок, яка завжди є лінійною. Це робить даний алгоритм більш доцільним у випадках, коли операції запису в пам’ять є ресурсомісткими.

Побудовані графіки підтвердили теоретичні оцінки складності алгоритмів, а також продемонстрували їхню поведінку залежно від типу вхідних даних.